

Estado inicial do trabalho

- Projeto base criado com a funcionalidade de cadastrar pet na main, pois o jaguara ainda não possui código definitivo inicial

```
PS C:\Users\miguel\Desktop\jaguara> npm run build

> jaguara@0.0.0 build
> tsc -b && vite build

vite v8.2.1 building client environment for production...
✓ 20 modules transformed.
computing gzip size...
dist/index.html      0.45 kB | gzip: 0.29 kB
dist/assets/index-DeCZBBIe.css 0.23 kB | gzip: 0.18 kB
dist/assets/index-CrJreMyB.js 192.71 kB | gzip: 60.87 kB

✓ built in 305ms
```

Commit 8601b70

MDrozino committed 10 minutes ago

criado projeto base Jaguara

main 0 parents commit 8601b70

23 files changed +3.151

Filter files...

- .gitignore
- README.md
- eslint.config.js
- index.html
- package-lock.json
- package.json
- public
 - favicon.svg
 - icons.svg
- src
 - App.tsx
- assets
 - hero.png
 - react.svg

Search within code

.gitignore +24

@@ -0,0 +1,24 @@

1 + # Logs

2 + logs

3 + *.log

4 + npm-debug.log*

5 + yarn-debug.log*

6 + yarn-error.log*

7 + pnpm-debug.log*

8 + lerna-debug.log*

9 +

10 + node_modules

11 + dist

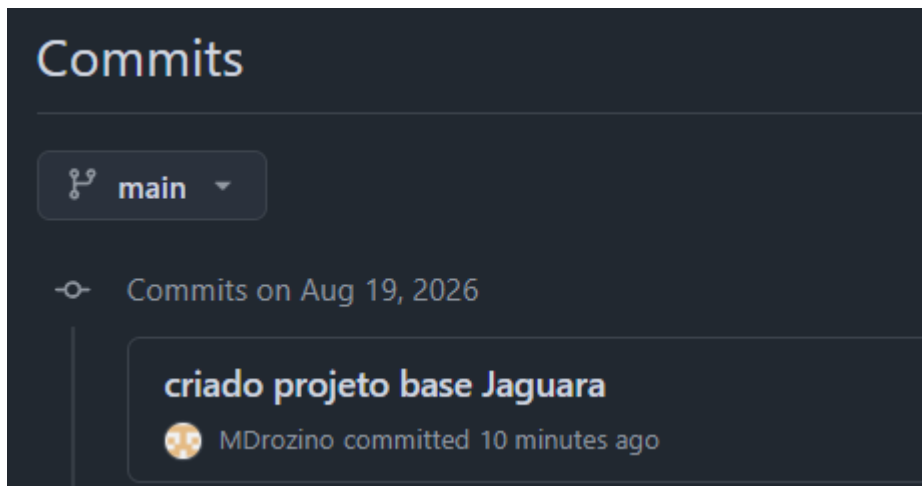
12 + dist-ssr

13 + *.local

14 +

15 + # Editor directories and files

16 + .vscode/*



Jaguara

Gerencie a rotina e os cuidados dos seus animais.

Cadastrar pet



Pets cadastrados

- **thor** - cao - sem raça - 1 kg

- Agora o experimento será na branch feature/medicamentos com Claude Code

Vamos usar few-shot porque:

O projeto tem padrões para cadastro de pets. Ao fornecer exemplos dessas implementações, o modelo vai identificar entidades, formulários, serviços e páginas estruturados e aplicar esses padrões à nova funcionalidade de medicamentos. Portanto, já sabemos o padrão que deve seguir, e daremos exemplos disso para a ia.

Exemplo:

O Jaguara representa entidades através de interfaces TypeScript.

Exemplo:

```
Pet {  
  - id  
  - name  
  - species  
  ...  
}
```

O formulário correspondente utiliza:

PetForm

A persistência utiliza:

petService

A página:

Pets

Siga esse padrão ao criar a entidade Medicacao.

System Prompt

Você é um desenvolvedor de software responsável por implementar uma funcionalidade no projeto Jaguara, uma aplicação para gerenciamento da rotina e dos cuidados de animais de estimação.

Sua tarefa é desenvolver a funcionalidade solicitada pelo usuário respeitando a arquitetura, tecnologias, padrões de código e convenções existentes no projeto.

Regras de desenvolvimento

1. Antes de implementar qualquer alteração, analise cuidadosamente o contexto e os arquivos fornecidos.
2. Não crie uma nova arquitetura ou padrão quando existir uma solução equivalente já utilizada no projeto.
3. Faça somente as alterações necessárias para implementar a funcionalidade solicitada.
4. Preserve as funcionalidades existentes e evite alterações em arquivos que não estejam relacionados à tarefa.
5. Utilize as mesmas tecnologias, convenções de nomenclatura, organização de arquivos e padrões de estilização encontrados no contexto fornecido.
6. Caso exista alguma informação necessária para a implementação que não esteja disponível no contexto, identifique a ausência antes de assumir uma solução.
7. Não invente APIs, componentes, funções ou estruturas que não estejam presentes no contexto sem deixar essa decisão explícita.
8. Mantenha o código simples, legível, modular e compatível com a arquitetura existente.

Objetivo da funcionalidade

A funcionalidade a ser desenvolvida neste trabalho é o registro de medicamentos associados a um animal de estimação.

O usuário deverá conseguir informar os dados necessários de um medicamento e associá-lo a um pet já cadastrado.

A implementação deve seguir os padrões existentes no projeto em relação a componentes, formulários, tipos, armazenamento de dados e interface.

Comportamento durante a implementação

Quando receber uma tarefa:

1. Analise os requisitos fornecidos.
2. Analise os arquivos disponibilizados como contexto.
3. Identifique os padrões relevantes existentes.
4. Planeje a alteração necessária.
5. Implemente somente o que foi solicitado.
6. Verifique se a implementação é consistente com o contexto fornecido.
7. Informe de forma objetiva quais arquivos foram alterados e o que foi implementado.

Não altere funcionalidades não relacionadas à tarefa.

Prompt que usaremos nos dois testes:

Implemente a funcionalidade de registro de medicamentos para um pet, seguindo os padrões de arquitetura, componentes, tipos, formulários, persistência e estilização já utilizados no projeto.

O que vai ser fornecido em cada teste:

Teste A (amplo) : system prompt + few-shot exemplos + feature prompt + arquivos contexto amplo

Teste B (curado): mesmo system prompt + mesmo few shot + mesmo feature prompt + arquivos relevantes apenas

Prompt teste A: Você é um desenvolvedor de software responsável por implementar uma funcionalidade no projeto Jaguar, uma aplicação para gerenciamento da rotina e dos cuidados de animais de estimação.

Sua tarefa é desenvolver a funcionalidade solicitada pelo usuário respeitando a arquitetura, tecnologias, padrões de código e convenções existentes no projeto.

Regras de desenvolvimento

- Antes de implementar qualquer alteração, analise cuidadosamente o contexto e os arquivos fornecidos.
- Não crie uma nova arquitetura ou padrão quando existir uma solução equivalente já utilizada no projeto.
- Faça somente as alterações necessárias para implementar a funcionalidade solicitada.
- Preserve as funcionalidades existentes e evite alterações em arquivos que não estejam relacionados à tarefa.

- Utilize as mesmas tecnologias, convenções de nomenclatura, organização de arquivos e padrões de estilização encontrados no contexto fornecido.
- Caso exista alguma informação necessária para a implementação que não esteja disponível no contexto, identifique a ausência antes de assumir uma solução.
- Não invente APIs, componentes, funções ou estruturas que não estejam presentes no contexto sem deixar essa decisão explícita.
- Mantenha o código simples, legível, modular e compatível com a arquitetura existente.

Objetivo da funcionalidade

A funcionalidade a ser desenvolvida neste trabalho é o registro de medicamentos associados a um animal de estimação.

O usuário deverá conseguir informar os dados necessários de um medicamento e associá-lo a um pet já cadastrado.

A implementação deve seguir os padrões existentes no projeto em relação a componentes, formulários, tipos, armazenamento de dados e interface.

Comportamento durante a implementação

Quando receber uma tarefa:

- Analise os requisitos fornecidos.
- Analise os arquivos disponibilizados como contexto.
- Identifique os padrões relevantes existentes.
- Planeje a alteração necessária.
- Implemente somente o que foi solicitado.
- Verifique se a implementação é consistente com o contexto - fornecido.
- Informe de forma objetiva quais arquivos foram alterados e o que foi implementado.

Não altere funcionalidades não relacionadas à tarefa.

Exemplo

O Jaguara possui uma entidade Pet definida em TypeScript:

```
```typescript
```

```
interface Pet {
```

```
 id: number;
```

```
name: string;
species: string;
breed: string;
birthDate: string;
weight: number;
}
```

O cadastro dessa entidade é realizado através de um componente `PetForm.tsx`.

A persistência é realizada através do serviço `petService.ts`.

A página `Pets.tsx` integra o formulário, o serviço e a listagem.

Ao implementar novas funcionalidades, siga esse mesmo padrão de separação entre tipos, componentes, serviços e páginas.

Implemente a funcionalidade de registro de medicamentos para um pet, seguindo os padrões de arquitetura, componentes, tipos, formulários, persistência e estilização já utilizados no projeto.

O medicamento deve possuir:

- nome;
- dosagem;
- frequência;
- horários;
- data de início;
- data de término;
- observações.

A implementação deve seguir os padrões existentes no projeto.

Não altere funcionalidades existentes que não sejam necessárias para implementar essa funcionalidade.

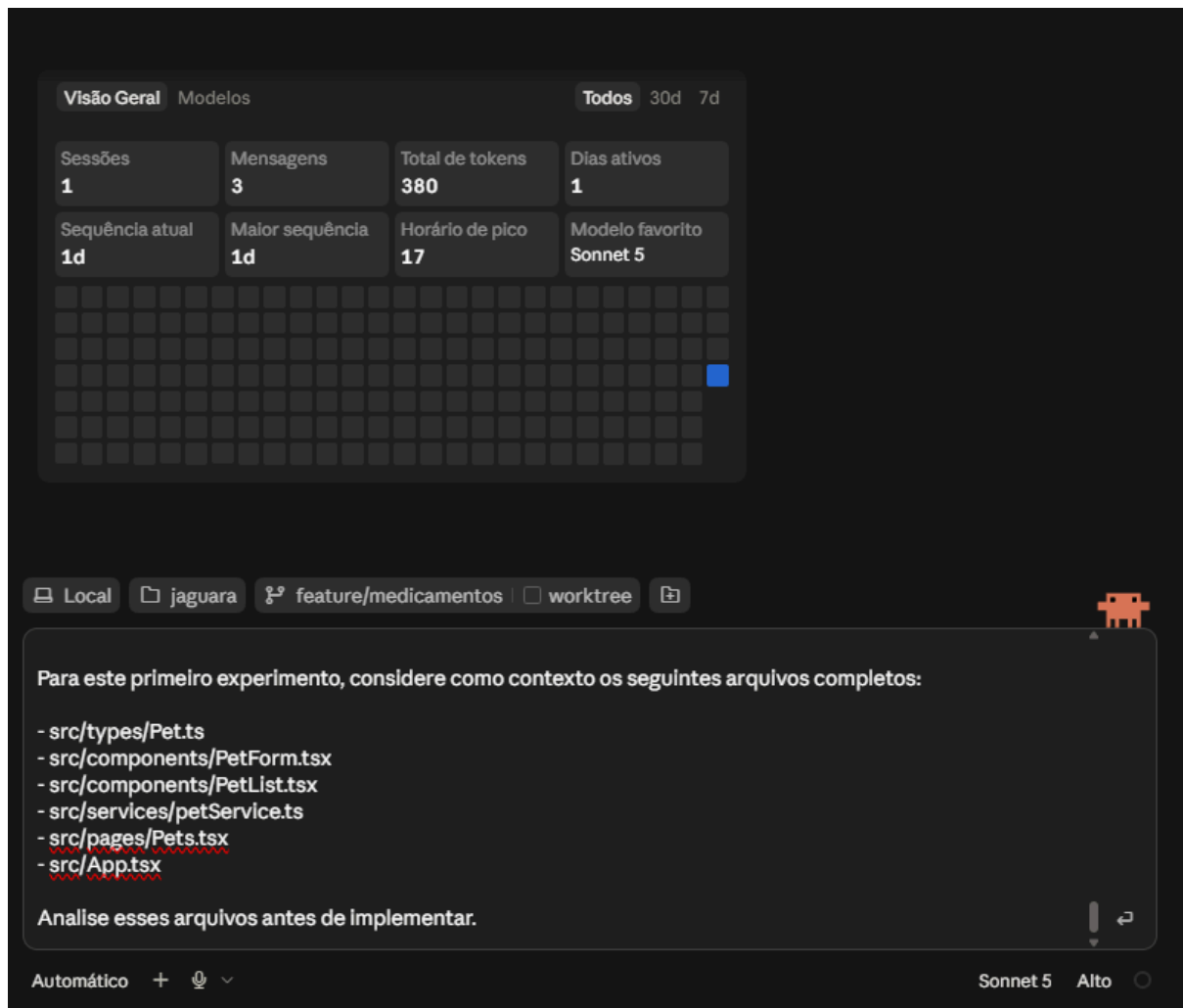
Ao finalizar, execute o build do projeto e corrija eventuais erros relacionados à implementação.

Para este primeiro experimento, considere como contexto os seguintes arquivos completos:

- `src/types/Pet.ts`

- src/components/PetForm.tsx
- src/components/PetList.tsx
- src/services/petService.ts
- src/pages/Pets.tsx
- src/App.tsx

Analise esses arquivos antes de implementar.



resultados teste A:



Espécie
Raça
dd/mm/aaaa
Peso
Cadastrar

### Pets cadastrados

- **thor** - cao - sem raça - 1 kg

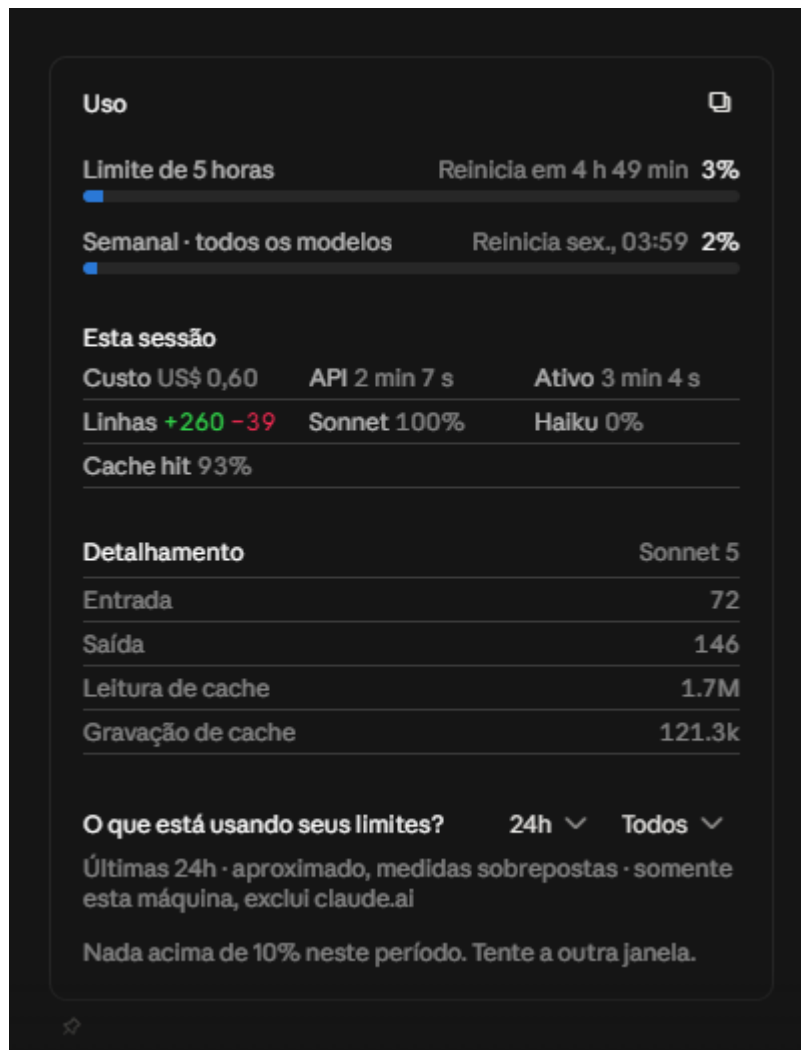
### Cadastrar medicamento

Selecione o pet
Nome do medicamento
Dosagem
Frequência
Horários (ex: 08:00, 20:00)
dd/mm/aaaa
dd/mm/aaaa
Observações
Cadastrar

### Medicamentos cadastrados

- **medicamento** - thor - 120mg - 2 vezes por dia - 8:00 e 18:00  
2026-08-01 até 2026-08-29  
obs

Tokens:



Teste B:

Você é um desenvolvedor de software responsável por implementar uma funcionalidade no projeto Jaguará, uma aplicação para gerenciamento da rotina e dos cuidados de animais de estimação.

Sua tarefa é desenvolver a funcionalidade solicitada pelo usuário respeitando a arquitetura, tecnologias, padrões de código e convenções existentes no projeto.

## Regras de desenvolvimento

- Antes de implementar qualquer alteração, analise cuidadosamente o contexto e os arquivos fornecidos.

- Não crie uma nova arquitetura ou padrão quando existir uma solução equivalente já utilizada no projeto.
- Faça somente as alterações necessárias para implementar a funcionalidade solicitada.
- Preserve as funcionalidades existentes e evite alterações em arquivos que não estejam relacionados à tarefa.
- Utilize as mesmas tecnologias, convenções de nomenclatura, organização de arquivos e padrões de estilização encontrados no contexto fornecido.
- Caso exista alguma informação necessária para a implementação que não esteja disponível no contexto, identifique a ausência antes de assumir uma solução.
- Não invente APIs, componentes, funções ou estruturas que não estejam presentes no contexto sem deixar essa decisão explícita.
- Mantenha o código simples, legível, modular e compatível com a arquitetura existente.

## ## Objetivo da funcionalidade

A funcionalidade a ser desenvolvida neste trabalho é o registro de medicamentos associados a um animal de estimação.

O usuário deverá conseguir informar os dados necessários de um medicamento e associá-lo a um pet já cadastrado.

A implementação deve seguir os padrões existentes no projeto em relação a componentes, formulários, tipos, armazenamento de dados e interface.

## ## Comportamento durante a implementação

Quando receber uma tarefa:

- Analise os requisitos fornecidos.
- Analise os arquivos disponibilizados como contexto.
- Identifique os padrões relevantes existentes.
- Planeje a alteração necessária.
- Implemente somente o que foi solicitado.
- Verifique se a implementação é consistente com o contexto - fornecido.
- Informe de forma objetiva quais arquivos foram alterados e o que foi implementado.

Não altere funcionalidades não relacionadas à tarefa.

## ## Exemplo

O Jaguar possui uma entidade Pet definida em TypeScript:

```
``typescript
interface Pet {
 id: number;
 name: string;
 species: string;
 breed: string;
 birthDate: string;
 weight: number;
}
```

O cadastro dessa entidade é realizado através de um componente PetForm.tsx.

A persistência é realizada através do serviço petService.ts.

A página Pets.tsx integra o formulário, o serviço e a listagem.

Ao implementar novas funcionalidades, siga esse mesmo padrão de separação entre tipos, componentes, serviços e páginas.

Implemente a funcionalidade de registro de medicamentos para um pet, seguindo os padrões de arquitetura, componentes, tipos, formulários, persistência e estilização já utilizados no projeto.

O medicamento deve possuir:

- nome;
- dosagem;
- frequência;
- horários;
- data de início;
- data de término;
- observações.

A implementação deve seguir os padrões existentes no projeto.

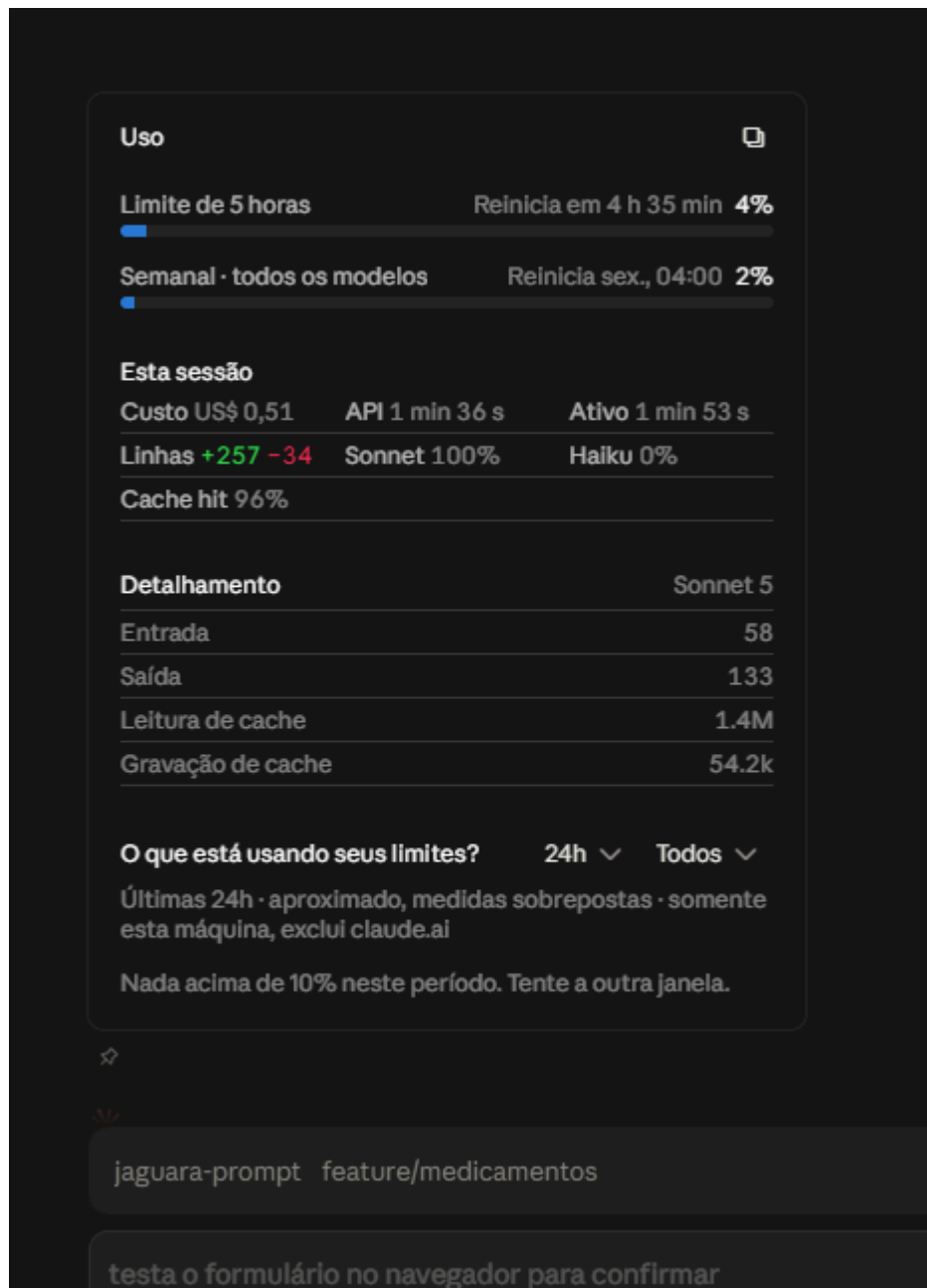
Não altere funcionalidades existentes que não sejam necessárias para implementar essa funcionalidade.

Ao finalizar, execute o build do projeto e corrija eventuais erros relacionados à implementação.

Para este primeiro experimento, considere como contexto os seguintes arquivos completos:

@src/types/Pet.ts @src/pages/Pets.tsx @src/components/PetForm.tsx  
@src/services/petService.ts @src/index.css

Analise esses arquivos antes de implementar.



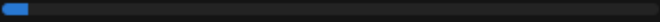
Mas não rodou... Para consertar custou mais tokens

Custo total:

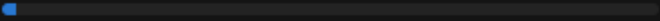
Uso



Limite de 5 horas      Reinicia em 4 h 30 min    4%



Semanal · todos os modelos      Reinicia sex., 03:59    2%



Esta sessão

Custo US\$ 1,09	API 3 min 25 s	Ativo 4 min 8 s
Linhas +290 -49	Sonnet 100%	Haiku 0%
Cache hit 98%		

Detalhamento	Sonnet 5
Entrada	122
Saída	463
Leitura de cache	3.4M
Gravação de cache	74.4k

O que está usando seus limites?    24h ▾    Todos ▾

Últimas 24h · aproximado, medidas sobrepostas · somente esta máquina, exclui claude.ai

Claude Browser MCP      18%

jaguara-prompt    feature/medicamentos